

B: Binary Dozens

Docenas binarias

Nombre del archivo fuente (*Source file name*): `bindozens.c`, `bindozens.cpp`, `bindozens.java` o `bindozens.py`

Autor (*Author*): Rodrigo Cardoso

Egan fundó recientemente una empresa de nano-huevos (*nano-eggs*): huevos microscópicos con prometedores usos farmacéuticos. Como los huevos suelen venderse por docenas, Egan también desarrolló las *nano-cajas* (*nano-crates*), cada una con capacidad para exactamente 12 nano-huevos. La empresa solo vende docenas completas: un número entero de nano-cajas, cada una completamente llena.

Producir huevos es costoso. Dado un inventario de H huevos, sea D el número de huevos que no caben en nano-cajas completas (de 12 huevos cada una). Estos huevos sobrantes deben descartarse.

Los siguientes ejemplos muestran algunos casos posibles:

- Si $H = 33$, se usan dos cajas y se descartan $D = 9$ huevos.
- Si $H = 48$, se usan cuatro cajas y se descartan $D = 0$ huevos.

Los volúmenes de producción pueden ser muy grandes, y la línea de producción usa un contador binario para registrar el tamaño de cada lote. Así, H en decimal puede expresarse en *binario* (*binary*) como B . Entonces, los ejemplos anteriores deben interpretarse así:

- Si $B = 100001$, entonces $D = 9$.
- Si $B = 110000$, entonces $D = 0$.

Ayuda a Egan escribiendo un programa que lea B e imprima D en decimal.

Entrada (Input)

La entrada consiste en varios casos de prueba, uno por línea. Cada caso de prueba es una cadena binaria B (formada solo por los caracteres `0` y `1`) que representa el número de nano-huevos H . Puedes suponer que $1 \leq |B| \leq 500$.

El fin de la entrada se indica con una línea que contiene un único asterisco (`*`).

La entrada debe leerse de la entrada estándar.

Salida (Output)

Para cada caso de prueba B , imprime el número D de nano-huevos sobrantes al empacarlos en nano-cajas.

La salida debe escribirse en la salida estándar.

Ejemplo de entrada (Sample Input)	Ejemplo de salida (Sample Output)
100001	9
110000	0
1	1
111	7
1010110	2
*	